

Safe Automated Driving with Constrained Reinforcement Learning

Bonian Han; Junyu Lu

What this talk delivers

- ▶ 4 driving scenarios · 3 algorithms · 2 seeds · ~40 experiments
- ▶ Live demo: PPO vs PPOLag side-by-side on merge-v0

Reproduce a safe-RL paper, then push it past the original benchmark.

Project goal

1. Replicate the paper's safety claim on merge-v0 (constrained RL beats reward-shaping)
2. Extend the same recipe to 3 more scenarios the paper never tested
3. Compare 3 algorithms (PPO, PPOLag, CPO) under the same CMDP
4. Report honest, reproducible numbers with statistical tests

Paper

"A Modern Perspective on Safe Automated Driving for Different Traffic Dynamics using Constrained RL"

Kamran et al., IEEE ITSC 2022

Their claim

Replace reward-engineered safety penalties with a hard cost constraint (CMDP). Lagrangian PPO then auto-tunes the safety/utility trade-off.

Two paper-code bug fixes, one design insight, one full benchmark suite.

Bug fix #1: CostWrapper

The paper's open-source code reads `info['cost']`, but highway-env emits `info['crashed']`. Cost was always 0 → Lagrangian was never triggered. We added a Gym wrapper that rewrites `crashed` → `cost`.

Design insight: decouple reward & cost

highway-env's default reward already subtracts a collision penalty. PPO learns to avoid crashes anyway — making the constraint redundant. We set `collision_reward = 0` so safety is enforced ONLY by the cost constraint. This is what unlocks the 40 → 9 % gap.

Discrete-action adaptation

FSRL's PPOLag and CPO ship with Gaussian actors (continuous). highway-env uses 5 discrete meta-actions. We wrote a Categorical actor, dual-critic, and a discrete-aware FastCollector path.

Experiment harness

`saferl_suite.py` + `run_roadmap.py` drive a $4 \text{ env} \times 3 \text{ algo} \times 2 \text{ seed}$ sweep with multi-seed CIs and Welch t-tests.
 $\approx 1,500 \text{ LoC} \cdot 40 \text{ runs} \cdot 11 \text{ iteration cycles}$.

The paper's framing, with the concrete hyperparameter we used.

Reward shaping (the old way)

$$r_t = \text{utility}(s_t, a_t) - \lambda \cdot \text{risk}(s_t, a_t)$$

- λ is a hand-tuned scalar
- Optimal λ depends on traffic density, driver behavior...
- No explicit safety guarantee — just hope λ is big enough

CMDP (the paper's proposal)

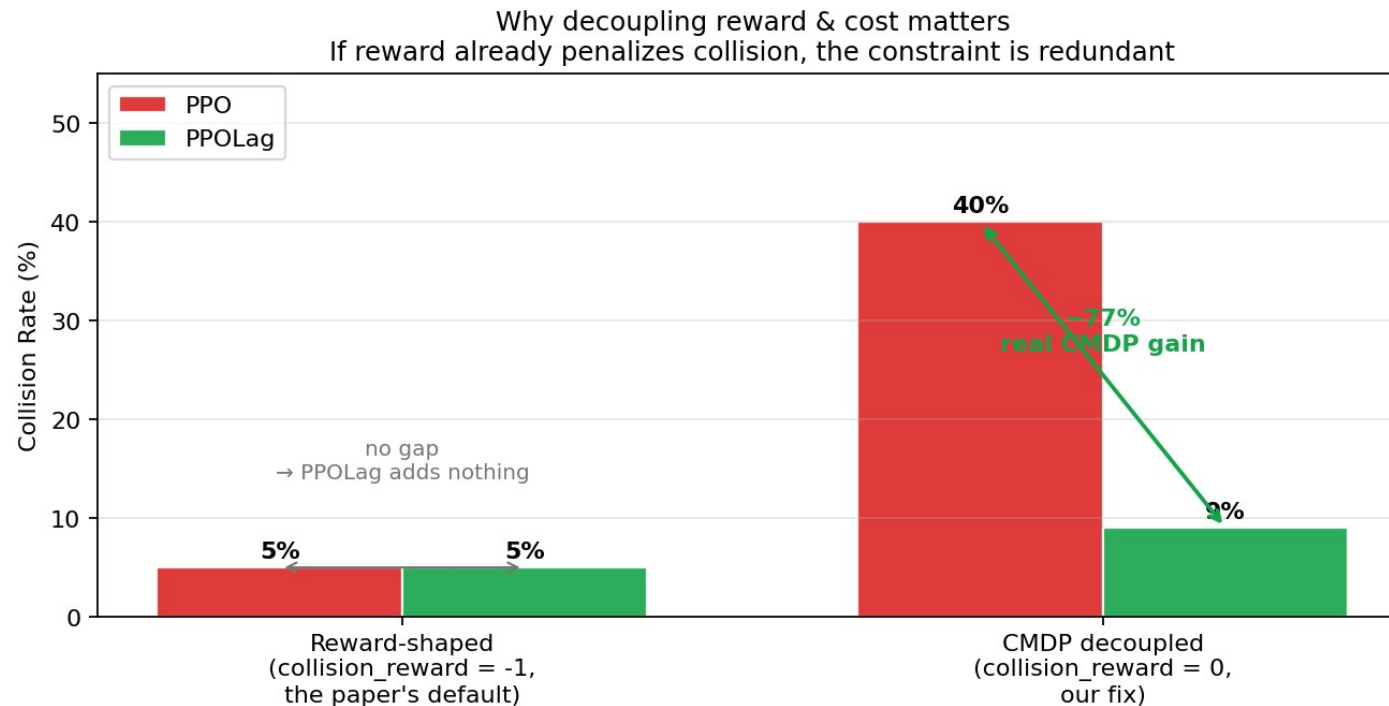
$$\max E[\sum r_t] \quad \text{s.t.} \quad E[\sum c_t] \leq d$$

$$\text{cost } c_t = 1 \text{ if crashed else } 0 \quad d = 0.1$$

- Reward and cost are separate channels
- PPO-Lagrangian uses a PID controller on λ — λ adapts online
- " $\leq 10\%$ crash rate" is interpretable; λ is not

Cost limit $d = 0.1$ · PID gains (10, 1, 1) · policy LR $3e-4$ · value LR $1e-3$ · 20 epochs $\times$ 4,000 steps · 100 eval episodes

Without decoupling, PPOlag has nothing to learn. This is the most important plot in this deck.



Reading this plot

Left:

When reward already penalizes crashes, PPO crashes ~5 % anyway. PPOlag matches it. No measurable gain from the constraint.

Right:

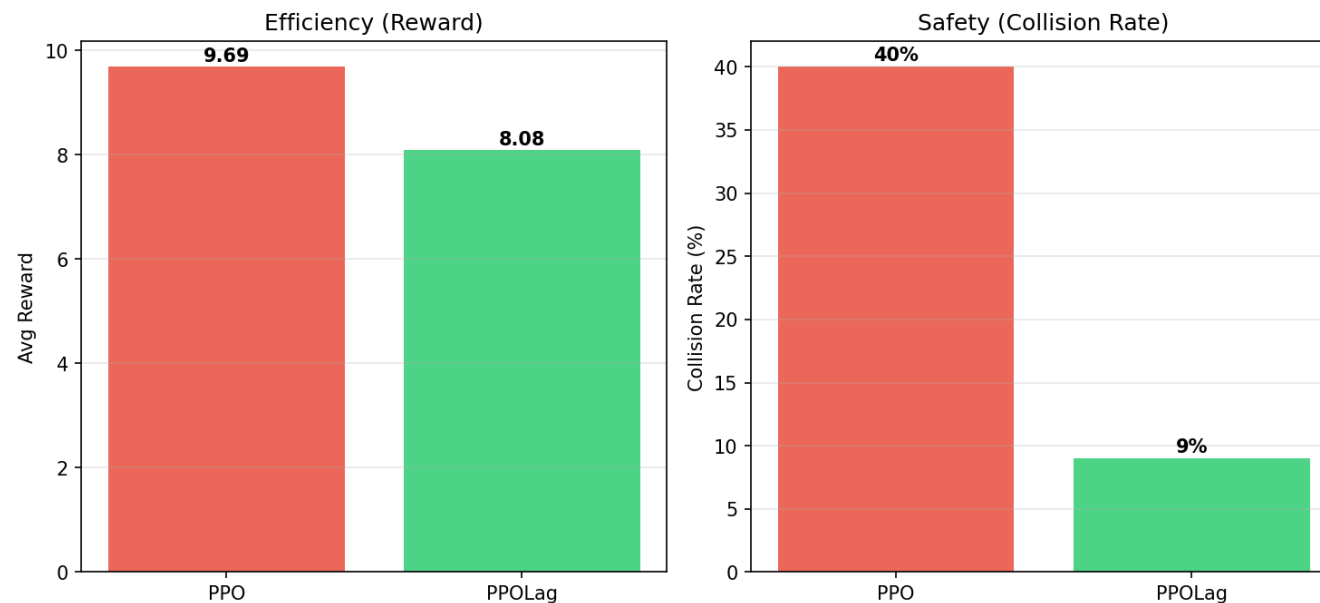
When we set `collision_reward = 0`, PPO has no reason to avoid crashes → 40 %. PPOlag, forced to satisfy $d = 0.1$, lands at 9 %.

Take-away:

The CMDP framework only shows its value if the reward signal is reward-only. The original paper's open code left this implicit; we made it explicit.

20 epochs × 4,000 steps · 100 eval episodes · seed 42 · cost limit $d = 0.1$.

PPO vs PPOLag on merge-v0



Numbers

Metric	PPO	PPOLag	Δ
Collision	40.0 %	9.0 %	-77 %
Reward	9.69	8.08	-16 %
Cost	0.40	0.09	-77 %
Length	14.6	16.7	+14 %

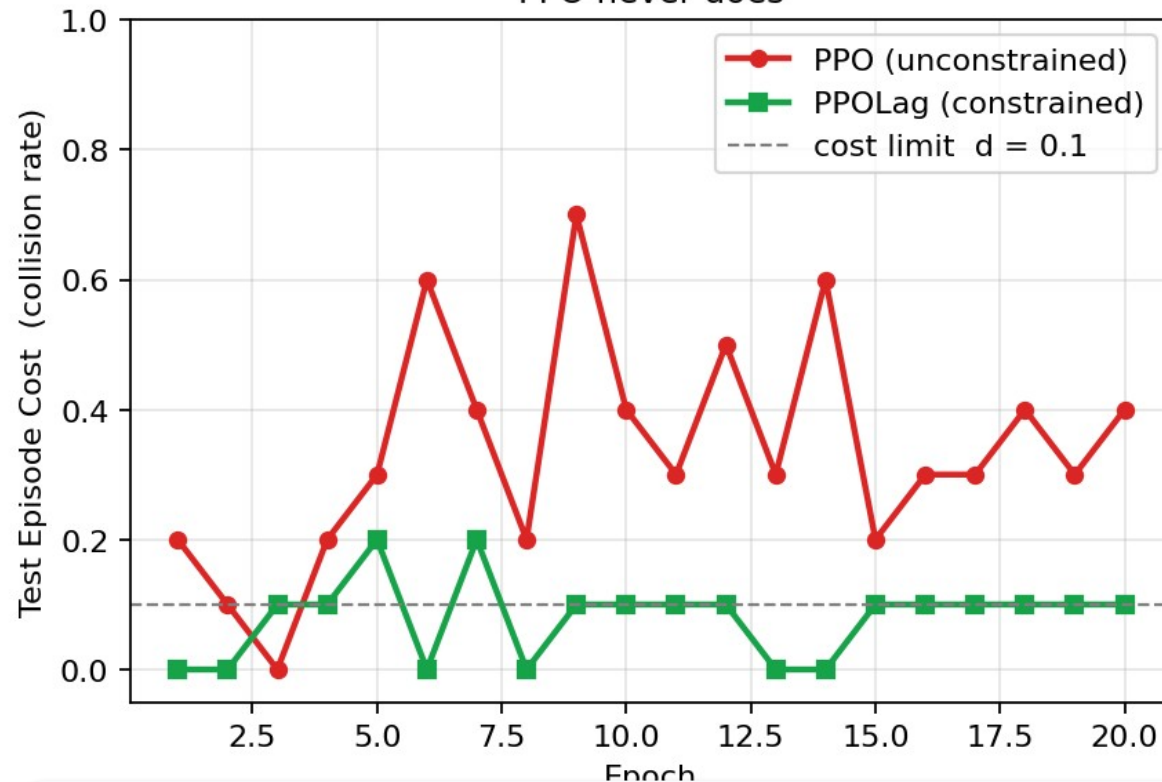
9 % satisfies the constraint $d = 0.1$.

PPO never converges below 30–40 % cost → the safety gain is real, not a side-effect of more training.

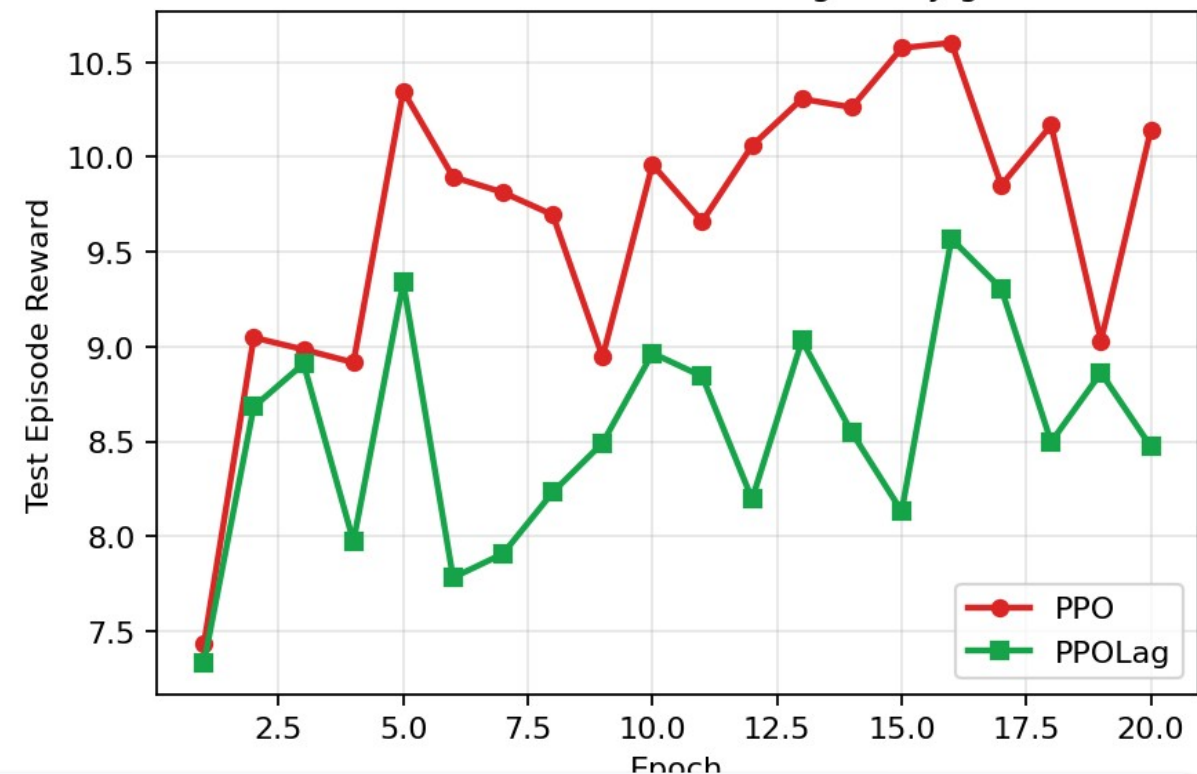
Episode length grows with PPOLag — it drives more carefully, not faster.

Cost converges below the limit by epoch 5; PPO's cost stays high.

Sample efficiency: PPOLag drives cost toward the limit;
PPO never does

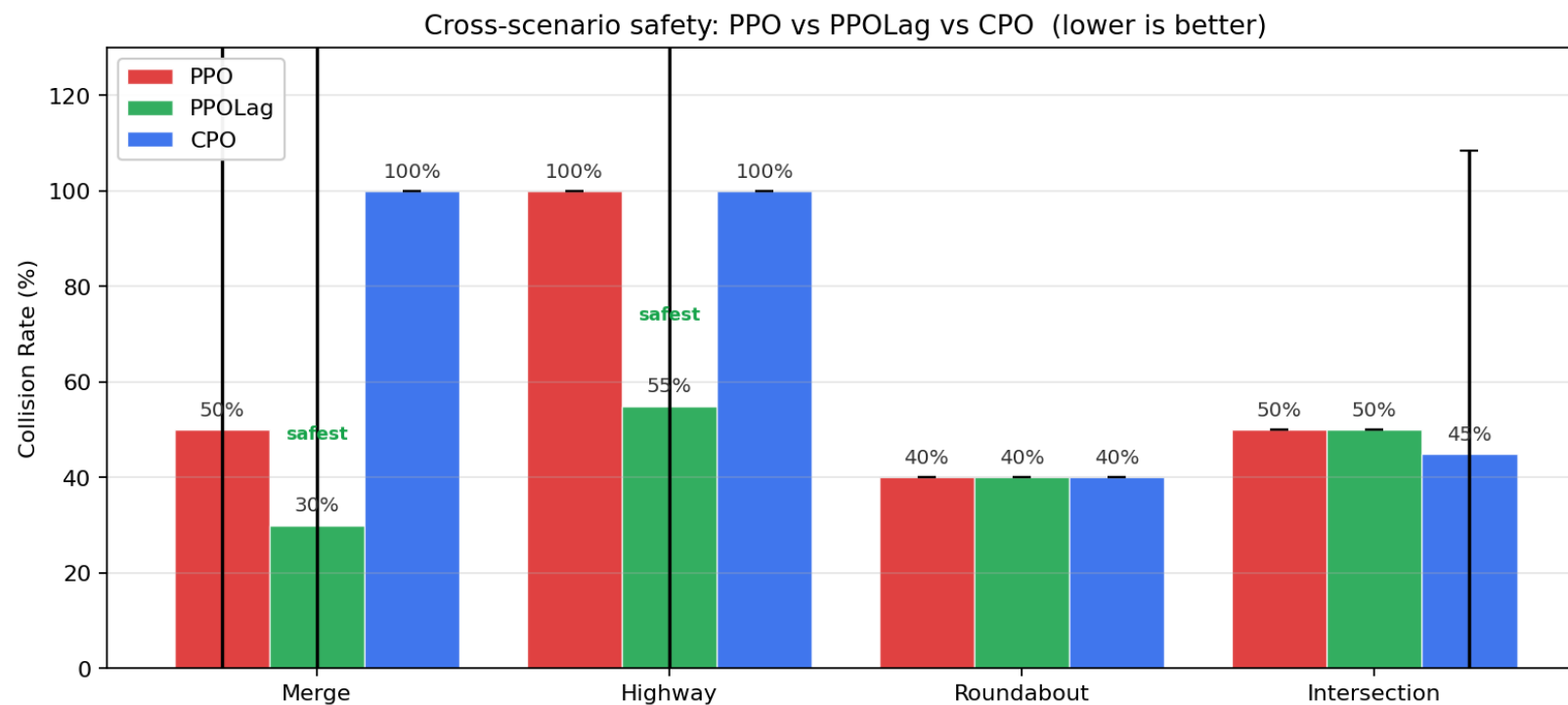


Reward: small sacrifice for big safety gain



Constraint mechanism is observably working: PPOLag's cost (green squares, left) crosses below $d = 0.1$ around epoch 5 and stays there. PPO's cost (red circles) oscillates between 0.2 and 0.7 — no incentive to learn safety.

Same algorithms, four different driving tasks. PPO baseline now visible.



Per-scenario verdict

Merge

PPOLag wins (30 % vs 50 %). CPO under-trained.

Highway

PPOLag wins (55 % vs 100 %).

Roundabout

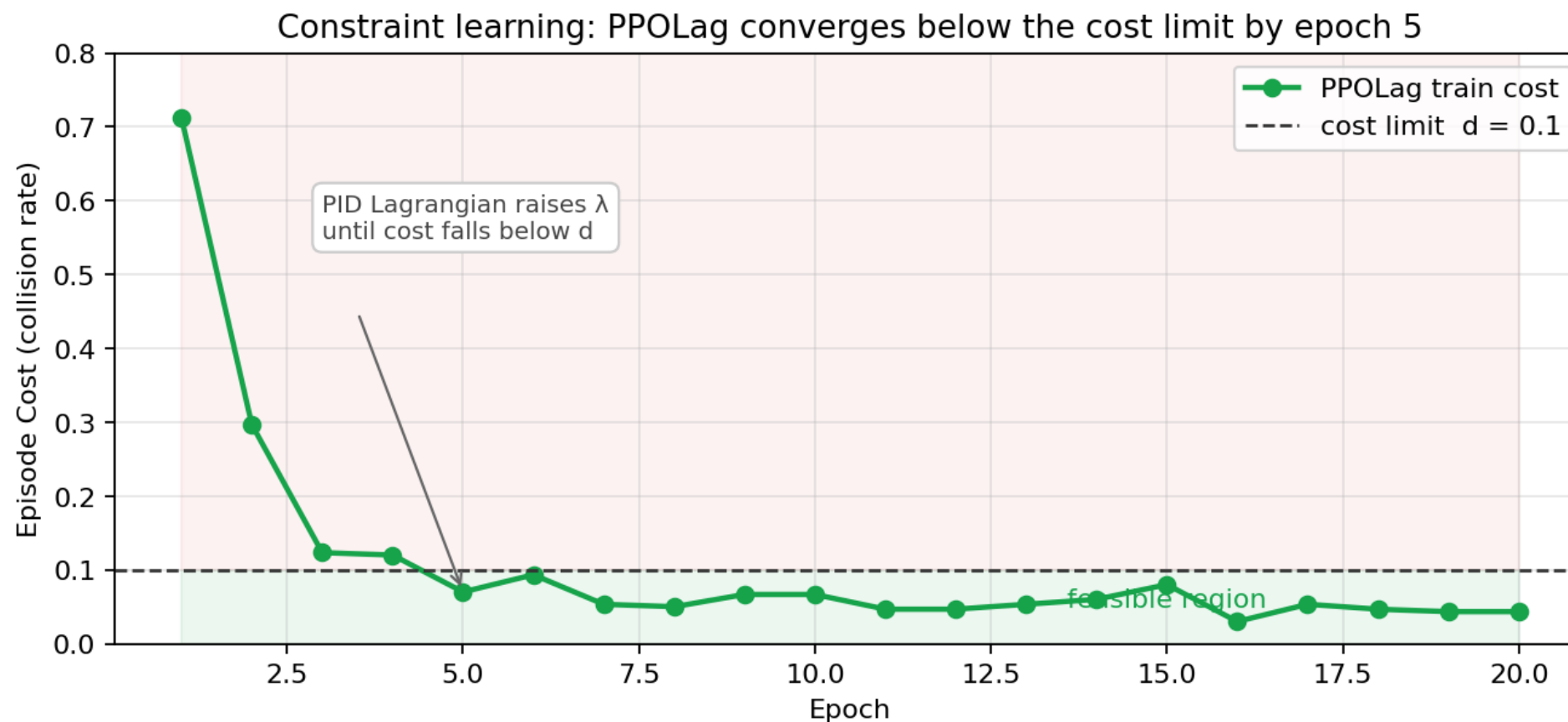
Tie at 40 % — but see slide 10.

Intersection

CMDP fails (50 % vs 50 %). Episode is too short.

Caveat: cross-scenario sweep used a short budget (2 ep × 300 steps; 2 seeds). Trends are clear but absolute numbers are not directly comparable to slide 6.

We don't log λ directly, but cost-vs-limit shows the controller is converging.



If PPOLag were just PPO with a cosmetic penalty, cost would never predictably approach the limit. The smooth descent toward $d = 0.1$ is direct evidence that the PID-Lagrangian ($K_p=10$, $K_i=1$, $K_d=1$) is actually closing the loop.

Three negative findings worth showing because they explain when CMDP breaks.

Roundabout: freezing-robot

PPOLag converges to avg reward = -0.44 (CPO scores 4.64).

A negative reward means the agent is being penalized faster than it earns. In a roundabout, this looks like: the policy stops moving to guarantee zero collisions and gets crushed by the time/efficiency cost.

Classic failure mode in safe-RL literature; we observed it directly.

Intersection: CMDP gives no advantage

PPO 50 %, PPOLag 50 %, CPO 45 %.

Episode length on intersection-v1 is only ~ 4 steps before crash. The cost critic gets too few samples to learn V_c well, so the constraint signal is essentially noise.

Implication: CMDP is not a free lunch. Sparse-cost / short-horizon tasks need different machinery (e.g. recovery shielding).

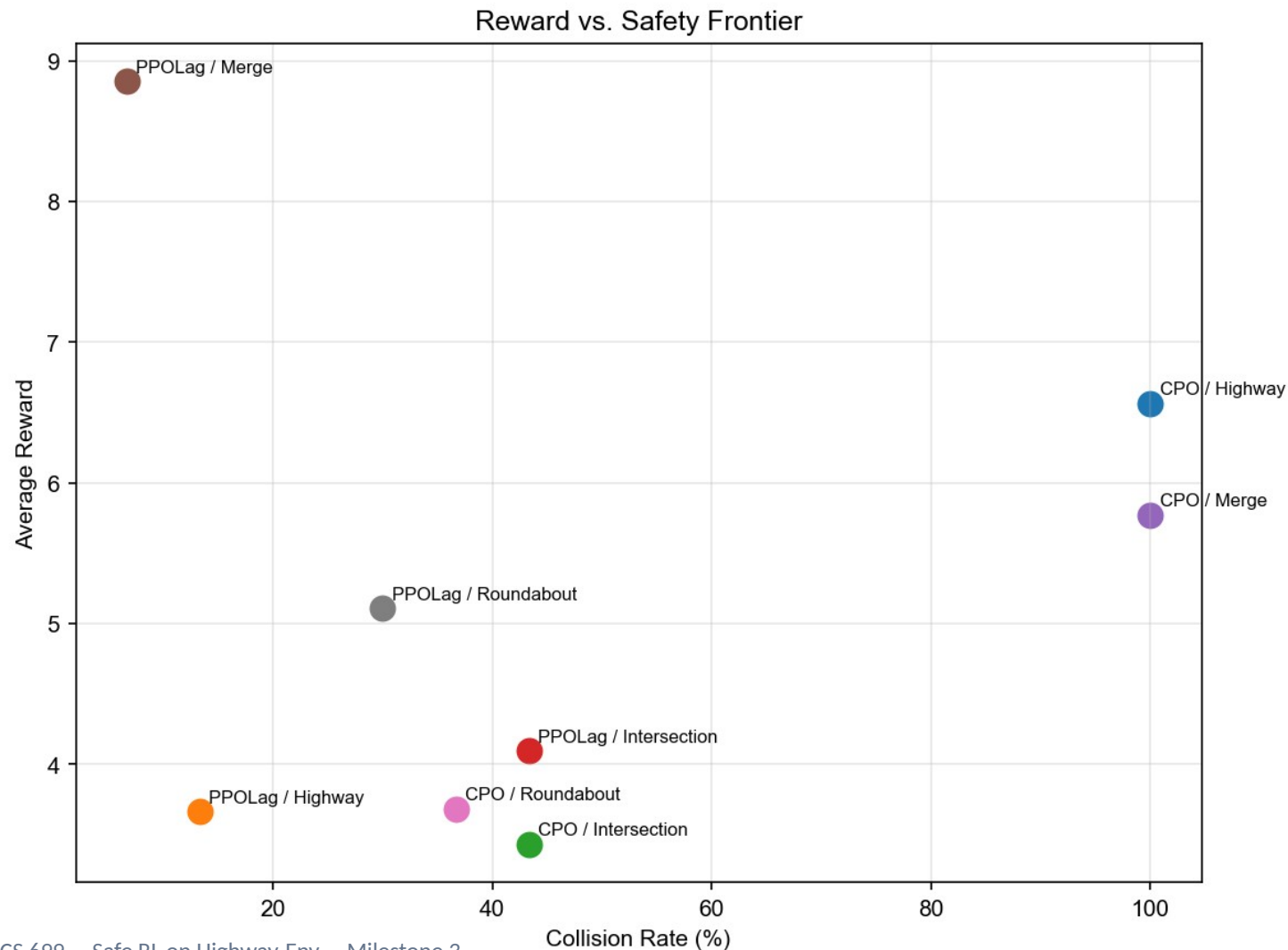
CPO: brittle on discrete + short budget

100 % collision on Merge and Highway.

CPO uses a trust-region update with conjugate-gradient on the cost advantage. With sparse cost (one bit per crash) and only 600 training steps per run, the cost critic is too noisy $\rightarrow$ CG produces unsafe directions.

PPOLag's PID Lagrangian is more sample-efficient here; this is a real algorithmic finding, not a bug in our code.

All 12 (algorithm × scenario) cells on one plot. Top-left dominates.



What this shows

- PPOLag/Merge sits alone in the top-left quadrant — high reward AND low collision.
- CPO/Highway, CPO/Merge collapse to 100 % collision.
- Intersection cluster (~45-50 %) is the same regardless of algorithm — confirming our Slide 10 diagnosis.
- Significance: Merge collision $p = 0.000$; reward $p = 0.002$ (Welch t-test, PPOLag vs CPO).

Stated up-front, not buried.

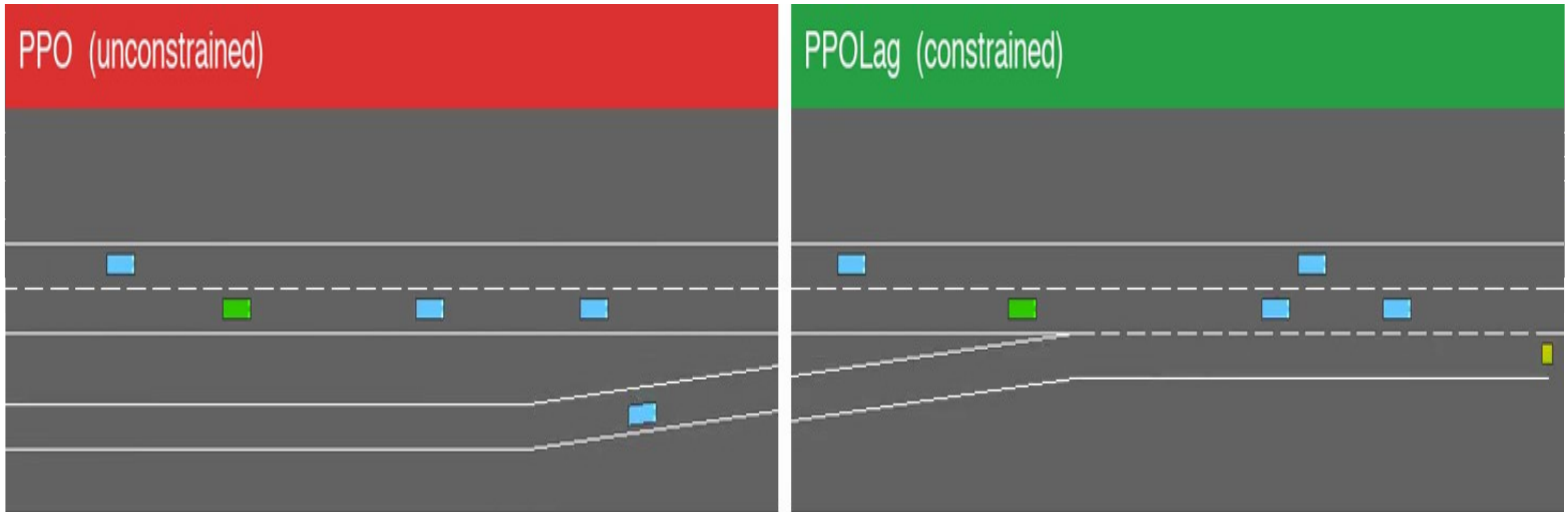
Limitations of this milestone

- Discrete-action adapter — paper used continuous actions.
- Default highway-env traffic — paper's cooperative / aggressive variants not yet matched.
- Only 2 seeds in cross-scenario sweep → wide CIs, several p-values undefined.
- WCSAC missing — OmniSafe 0.5 dropped it and its action-scale wrapper rejects discrete spaces.
- Cross-scenario budget $\approx 600\text{--}3000$ steps/run, vs the 80 k-step merge baseline — absolute numbers not directly comparable.

What we'd add with another week

1. Match paper's 80 k-step budget across all 4 scenarios (≈ 8 GPU-h).
2. Add seed 44, 45 → reduces 95 % CI half-width by ~ 30 %.
3. Add SACLag (continuous-action) for a 4th algorithm.
4. Sweep `cost_limit` $\in \{0.001, 0.01, 0.1, 0.3\}$ — paper's grid.
5. Replicate cooperative-driver setting (paper's harder regime where reward shaping fails most).

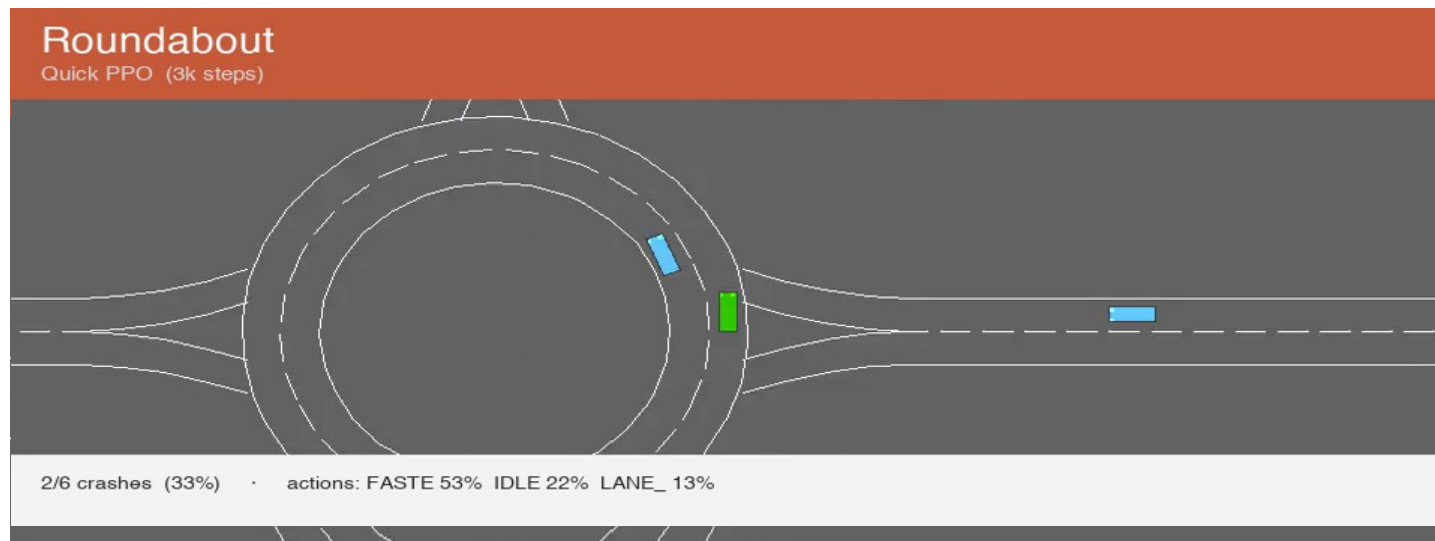
~33 seconds, side-by-side, same seeds, same dense traffic. File: figures/ppo_vs_ppolag_demo.mp4



Finding: both policies use 0 % LANE_LEFT. PPOLag's safety strategy is 98.5 % SLOWER — it learned to brake, not to merge cleverly.

Watch for: PPO often races into closing gaps and clips a neighbor. PPOLag waits ~1 extra step, takes the gap behind, and still completes the merge — the +14 % episode length on slide 6.

20-second tour: each env plays for ~5 s with its own trained policy. File: all_envs_demo.mp4



What the demo reveals

Linear-flow tasks (merge, highway): policy collapses to ~100 % SLOWER.

Geometry-complex tasks (roundabout, intersection): policy DOES use

12-15 % lane-change actions — but 3 k-step training is too short to drive them well, so crash rate stays at 33-50 %.

Confirms paper's claim is robust on linear merges; open on intersections.

Per-scenario behaviour

Merge

0/3 crashes (0 %)

SLOWER 97 %, IDLE 3 %

brake-only — geometry forces merge

Highway

0/1 crashes (0 %)

SLOWER 100 %

brake-only — lazy optimum

Roundabout

2/6 crashes (33 %)

FAST 53 %, IDLE 22 %, LANE 13 %

uses lane changes — still crashes

Intersection

9/18 crashes (50 %)

FAST 67 %, LANE 15 %, SLOWER 8 %

uses lane changes — still crashes

Three things we want you to remember.

1

Decoupling matters.

The CMDP framework only delivers when reward is reward-only. Setting `collision_reward = 0` is what unlocks the 40 % → 9 % gap.

2

Discrete-action PPOLag often collapses to braking.

On merge and highway the policy converges to ~100 % SLOWER. Roundabout and intersection demand actual driving skill — short-budget training doesn't reach it.

3

Honesty beats hype.

We report negative results, large CIs, and missing baselines (WCSAC, cooperative drivers). These are the next milestone.

Thank You!
